

Tối ưu hóa Truy vấn Phân tán bằng Semi-Join

1. Giới thiệu

Tài liệu này phác thảo thiết kế và cơ sở lý thuyết cho việc triển khai phép toán Semi-Join phân tán ($R \bowtie S$). Dự án giải quyết bài toán giảm thiểu chi phí truyền tải mạng trong quá trình xử lý truy vấn phân tán, một chủ đề cốt lõi được đề cập trong cuốn sách *Principles of Distributed Database Systems* của M. Tamer Özsu và Patrick Valduriez.

Hệ thống của tôi minh họa điều này bằng cách đánh giá một truy vấn trải dài qua nhiều node (điểm nút): hai node lưu trữ bảng Employees (R) và một node lưu trữ bảng Assignments (S). Mục tiêu là tìm tất cả các nhân viên hiện đang tham gia ít nhất một dự án.

2. Kiến trúc và các thành phần

Hệ thống được xây dựng trên một kiến trúc mô phỏng gồm 4 node sử dụng Python và FastAPI (3 Node Dữ liệu + 1 Node Điều phối):

- **Node 1 (Employee Site A):** Quản lý nửa đầu của tập dữ liệu Employees được phân mảnh ngang (5.000 bản ghi). Cung cấp các API REST để truy xuất toàn bộ dữ liệu và lọc theo Semi-Join.
- **Node 3 (Employee Site B):** Quản lý nửa sau của tập dữ liệu Employees (5.000 bản ghi). Cung cấp các API tương tự như Node 1.
- **Node 2 (Assignment Site):** Quản lý tập dữ liệu Assignments (50.000 bản ghi). Cung cấp các API REST để truy xuất toàn bộ dữ liệu hoặc chỉ các khóa ngoại duy nhất.
- **Coordinator (Node Điều phối):** Điều phối việc thực thi truy vấn trên 3 node dữ liệu, đo lường kích thước payload và tính toán tổng thời gian thực thi.

Lý do sử dụng các API HTTP hạng nhẹ (FastAPI) và Pandas DataFrames là để mô phỏng giao tiếp vi dịch vụ (microservices) trong thế giới thực, đồng thời giữ cho các thao tác I/O chạy nhanh trên bộ nhớ RAM, cho phép chúng tôi quan sát chính xác sự khác biệt về độ trễ và băng thông mạng.

3. Logic xử lý Truy vấn Phân tán (DQP)

Theo Özsu và Valduriez, các truy vấn phân tán có thể được tối ưu hóa bằng cách giảm kích thước của các quan hệ trung gian được gửi qua mạng.

3.1. Standard Join (Join truyền thống - Ship-Whole-Table)

Trong phương pháp Standard Join nguyên thủy, Node 2 gửi toàn bộ bảng Assignments đến Coordinator, tiếp theo Node 1 và Node 3 gửi toàn bộ bảng Employees đến Coordinator.

- Số Byte truyền tải: $\text{Size}(S) + \text{Size}(R_1) + \text{Size}(R_3)$
- Nhược điểm: Chi phí truyền tải cực kỳ cao, đặc biệt là khi S có kích thước lớn.

3.2. Semi-Join ($R \bowtie S$)

Phương pháp Semi-Join giúp giảm thiểu lượng dữ liệu truyền tải và tận dụng sức mạnh xử lý song song:

1. Phép chiếu (Projection): Node 2 chiếu ra cột thuộc tính dùng để Join (EmpID) từ bảng S và loại bỏ các giá trị trùng lặp, tạo thành tập S'.
 2. Truyền tải (Transfer): Node 2 gửi S' cho Coordinator. Coordinator sau đó phát sóng (broadcast) tập này cho cả Node 1 và Node 3.
 3. Lọc (Filtering): Node 1 và Node 3 lọc dữ liệu R cục bộ của mình dựa trên tập S', tạo ra R'_1 và R'_3 một cách song song.
 4. Kết quả (Result): Node 1 và Node 3 gửi kết quả ngược lại cho Coordinator để hợp nhất (Union).
- Số Byte truyền tải: $\text{Size}(S) + \text{Size}(R'_1) + \text{Size}(R'_3) + \text{Chi phí Request}$
 - Ưu điểm: Bằng cách đánh đổi một lượng nhỏ công việc xử lý tại Node 2 (phép chiếu và loại bỏ trùng lặp), chúng ta tránh được việc phải gửi các cột dữ liệu khổng lồ không cần thiết của bảng S.

4. Phân tích mô hình chi phí

Trong môi trường phân tán, tổng chi phí (Total_Cost) là sự kết hợp tuyến tính của chi phí I/O, CPU và Giao tiếp mạng (Communication):

$$\text{Total_Cost} = (C_{cpu} \times \#inst) + (C_{io} \times \#ios) + (C_{msg} \times \#msgs) + (C_{tr} \times \#bytes)$$

Vì các tập dữ liệu đều nằm trong bộ nhớ RAM (bỏ qua C_{io}) và số lượng tin nhắn (C_{msg}) là nhỏ và cố định cho cả hai phương pháp, yếu tố quyết định lớn nhất trở thành $C_{tr} \times \#bytes$.

Đánh giá trên lý thuyết: Giả sử $\text{Card}(S) = 50,000$ và $\text{Card}(S') = 7,985$. Giả sử kích thước của một tuple Assignment là 80 byte và một EmpID là 10 byte:

- Khối lượng payload gửi đi bằng Standard Join: $50,000 \times 80 = 4,000,000$ byte.
- Khối lượng payload gửi đi bằng Semi-Join: $7,985 \times 10 \approx 79.850$ byte.

Sự giảm sút khổng lồ về số byte truyền tải mạng này dẫn đến Hệ số giảm kích thước (Size Reduction Factor) lên tới ~77% cho toàn bộ truy vấn, qua đó chứng minh trực tiếp nhận định của Özsu rằng Semi-Join cực kỳ hiệu quả trong việc giảm thiểu độ trễ giao tiếp của Hệ quản trị CSDL Phân tán.

5. Xử lý lỗi (Độ tin cậy)

Coordinator của dự án đã triển khai một cơ chế timeout cơ bản. Trong một giao dịch phân tán, nếu một site tham gia gặp sự cố (ví dụ: Node 2 bị sập hoặc ngắt kết nối), Coordinator sẽ phát hiện endpoint không thể truy cập thông qua lỗi ConnectionError và an toàn hủy bỏ giao dịch (Abort) thay vì bị treo vô thời hạn. Điều này chứng tỏ khả năng chịu lỗi (Fault Tolerance) nền tảng của hệ thống.